

Deep Learning

Treinamento de LLMs: do Pré-treinamento ao Alinhamento

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

abril de 2026

Visão Geral

1. Pré-treinamento
2. Fine-tuning Supervisionado (SFT)
3. RLHF com PPO
4. Direct Preference Optimization (DPO)
5. Outros Métodos de Alinhamento
6. Ajuste Eficiente em Parâmetros (PEFT)
7. Quantização
8. Exercícios Práticos: PEFT e Quantização

Overview

1. Pré-treinamento
2. Fine-tuning Supervisionado (SFT)
3. RLHF com PPO
4. Direct Preference Optimization (DPO)
5. Outros Métodos de Alinhamento
6. Ajuste Eficiente em Parâmetros (PEFT)
7. Quantização
8. Exercícios Práticos: PEFT e Quantização

Objetivo: *Next-Token Prediction*

O pré-treinamento de LLMs se baseia em **modelagem de linguagem causal**: dado um prefixo de tokens, prever o próximo.

Função de perda — *Language Modeling*

$$\mathcal{L}_{\text{LM}}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_1, \dots, x_{t-1})$$

Esta perda é equivalente à **entropia cruzada** entre a distribuição real (token correto) e a distribuição prevista pelo modelo.

Propriedade chave: o corpus de texto é a própria supervisão — não há necessidade de anotação humana.

Exemplo de sequência de treino

Contexto	→	Alvo
0 gato	→	sentou
0 gato sentou	→	no
0 gato sentou no	→	tapete

Uma única sequência de comprimento T gera $T-1$ pares (contexto, alvo) de forma eficiente com *teacher forcing*.

Datasets de Pré-treinamento

Grandes corpora públicos

- **Common Crawl:** rastreamento da web, bruto e ruidoso
- **C4** (*Colossal Clean Crawled Corpus*): versão filtrada do CC
- **The Pile:** 825 GB de texto diversificado (livros, código, arXiv, GitHub)
- **ROOTS** (BigScience): corpus multilíngue (46 idiomas, 1.6 TB)
- **FineWeb** (HuggingFace): 15 T de tokens filtrados da web

Etapas de processamento

1. **Filtragem:** remoção de conteúdo de baixa qualidade, spam e conteúdo adulto (heurísticas + classificadores)
2. **Deduplicação:** remoção de documentos duplicados ou quase-duplicados (MinHash, suffix arrays)
3. **Tokenização:** conversão para tokens (ver Aula 42)
4. **Shuffling:** embaralhamento para evitar correlações temporais no gradiente

Tokenização e Vocabulário

A tokenização converte texto bruto em sequências de **inteiros** consumíveis pelo modelo (ver Aula 42 para detalhes completos).

Tamanho do vocabulário e impacto

- Vocabulário menor $\Rightarrow$ sequências mais longas, mais computação
- Vocabulário maior $\Rightarrow$ matriz de embedding maior, cobertura melhor
- Modelos modernos: 32 k – 200 k tokens (BPE ou Unigram)

O *tokenizer* é **treinado antes** do modelo, no mesmo corpus, e permanece fixo durante o treinamento.

Comparação de vocabulários

Modelo	Vocab	Algoritmo
GPT-2	50 257	BPE
LLaMA-2	32 000	BPE (sentencepiece)
LLaMA-3	128 256	BPE
BERT	30 522	WordPiece
T5	32 100	Unigram

Referência cruzada: **Aula 42** —
Tokenização e Vocabulário.

Scaling Laws (Chinchilla)

Hoffmann et al. (2022) mostraram que modelos anteriores eram **sub-treinados em dados** em relação ao seu tamanho.

Lei de Chinchilla

Para um orçamento computacional C fixo, o número ótimo de parâmetros N^* e de tokens de treino D^* satisfazem:

$$N^* \propto \sqrt{C}, \quad D^* \propto \sqrt{C} \quad \implies \quad D^* \approx 20 \cdot N^*$$

Conclusão prática: para um modelo de 7B parâmetros, treinar em $\sim 140\text{B}$ tokens é o ponto ótimo de custo-benefício.

GPT-3 (175B params, 300B tokens) e PaLM (540B params, 780B tokens) foram treinados longe do ponto ótimo.

Recomendações de escala

Parâmetros	Tokens (Chinchilla)	Tokens (Prática)
1B	20B	100B+
7B	140B	1T+
13B	260B	1T+
70B	1.4T	2T+

Na prática, modelos são treinados com mais tokens do que o ótimo de Chinchilla para melhorar inferência (ex.: LLaMA).

Estágios do Pré-treinamento

1. *Warm-up* do LR

- LR cresce linearmente de 0 até $\eta_{\max}$
- Dura tipicamente 1–2% do treino total
- Evita explosão de gradiente no início
- Estabiliza a norma dos parâmetros

2. *Cosine Decay*

- LR decai de $\eta_{\max}$ a $\eta_{\min}$
- Seguindo um cosseno:

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \times \left(1 + \cos \frac{\pi t}{T}\right)$$

- Transição suave evita instabilidades
- Amplamente adotado (GPT, LLaMA)

3. *Batch Ramp-up*

- *Batch size* cresce ao longo do treino
- Início: mini-batches pequenos para convergência rápida
- Fim: batches grandes para melhor generalização
- Reduz o número de passos sem perder qualidade
- Usado no GPT-3 e PaLM

Overview

1. Pré-treinamento
- 2. Fine-tuning Supervisionado (SFT)**
3. RLHF com PPO
4. Direct Preference Optimization (DPO)
5. Outros Métodos de Alinhamento
6. Ajuste Eficiente em Parâmetros (PEFT)
7. Quantização
8. Exercícios Práticos: PEFT e Quantização

SFT: Ajuste para Seguir Instruções

Após o pré-treinamento, o modelo sabe “completar texto”, mas não necessariamente **seguir instruções** de forma útil.

O *Supervised Fine-Tuning* (SFT), ou *Instruction Tuning*, ajusta o modelo em pares (instrução, resposta) de alta qualidade.

Função de perda do SFT

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t \in \mathcal{R}} \log P_{\theta}(x_t \mid x_{<t})$$

onde $\mathcal{R}$ são **apenas os tokens da resposta** (não do prompt/instrução).

Mascarar o prompt evita que o modelo “memorize” as instruções em vez de aprender a

Exemplo de amostra de SFT

[INSTRUÇÃO — mascarado]
Explique o que é retropropagação.

[RESPOSTA — perda calculada aqui]
Retropropagação é o algoritmo que calcula gradientes...

Datasets para Instruction Tuning

Datasets clássicos

- **FLAN** (Chung et al., 2022): 1.8k tarefas NLP formatadas como instruções; modelo base de até 540B parâmetros (PaLM); mostrou que diversidade de tarefas > volume
- **ShareGPT**: conversas reais compartilhadas por usuários do ChatGPT; cobertura ampla, qualidade variável
- **LIMA** (Zhou et al., 2023): apenas 1.000 exemplos **muito bem curados** — quase igual a 52k exemplos do Alpaca

Quantidade vs. Qualidade

Dataset	Exemplos	Qualidade
Alpaca (GPT-3.5)	52 000	Baixa–Média
ShareGPT	90 000	Média
LIMA	1 000	Alta
OpenHermes-2.5	1 000 000	Alta

Lição do LIMA: “1000 exemplos cuidadosamente selecionados superam 52000 gerados automaticamente.”

Chat Templates e Tokens Especiais

Modelos de chat usam **tokens especiais** para delimitar turnos da conversa, permitindo multi-turno e controle de papéis.

Tokens comuns

- `<|im_start|>`, `<|im_end|>` (ChatML / Qwen)
- `[INST]`, `[/INST]` (LLaMA-2 chat)
- `<|start_header_id|>` (LLaMA-3)
- `<s>` (BOS), `</s>` (EOS)

O template define **onde calcular a perda**: apenas nos tokens gerados pelo assistente.

Exemplo: formato ChatML

```
<|im_start|>system
Você é um assistente útil.
<|im_end|>
<|im_start|>user
Explique redes neurais.
<|im_end|>
<|im_start|>assistant
[perda calculada aqui]
Redes neurais são...
<|im_end|>
```

SFT vs. Pré-treinamento: Diferenças de Otimização

Pré-treinamento

- LR alto: $10^{-4} - 3 \times 10^{-4}$
- Centenas de bilhões de tokens
- *Batch size* muito grande (milhões de tokens)
- *Weight decay*: 0.1
- Foco: **absorver conhecimento**

Fine-tuning (SFT)

- LR baixo: $10^{-6} - 2 \times 10^{-5}$
- Milhares a milhões de exemplos
- *Batch size* menor (128–512 sequências)
- Poucas épocas (1–3) para evitar *overfitting*
- Foco: **adaptar comportamento**

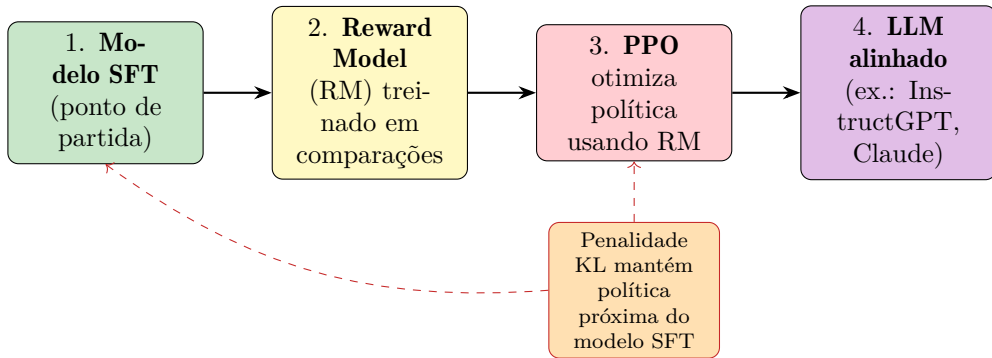
Risco de *Catastrophic Forgetting*

LR alto ou muitas épocas podem fazer o modelo “esquecer” conhecimento adquirido no pré-treinamento. Use *learning rate* baixo e monitore *benchmarks* gerais ao longo do SFT.

Overview

1. Pré-treinamento
2. Fine-tuning Supervisionado (SFT)
- 3. RLHF com PPO**
4. Direct Preference Optimization (DPO)
5. Outros Métodos de Alinhamento
6. Ajuste Eficiente em Parâmetros (PEFT)
7. Quantização
8. Exercícios Práticos: PEFT e Quantização

Pipeline Completo do RLHF



Etapa 1: amostrar pares de respostas do modelo SFT

Etapa 2: humanos escolhem a resposta preferida

Etapa 3: treinar o RM com *Bradley-Terry*

Etapa 4: usar PPO para otimizar a política

maximizando recompensa do RM

com penalidade KL em relação ao SFT

Treinamento do *Reward Model*

O *Reward Model* (RM) é treinado para **prever preferência humana** entre pares de respostas para o mesmo prompt.

Coleta de dados: para cada prompt x , amostram-se duas respostas (y_w, y_l) onde y_w é a preferida pelos anotadores.

Função de perda do RM (Bradley-Terry)

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \right]$$

onde $r_\phi(x, y)$ é o *scalar reward* predito.

Datasets de comparações humanas

Dataset	Pares	Fonte
HH-RLHF (Anthropic)	170k	Humanos
WebGPT Comparisons	19k	Humanos
SHP	385k	Reddit
UltraFeedback	64k	GPT-4

O RM geralmente parte do **mesmo modelo SFT** com uma cabeça de regressão escalar adicionada.

Função Objetivo do PPO com Penalidade KL

O PPO otimiza a política π_θ para maximizar recompensa enquanto permanece próximo da política SFT de referência π_{ref} .

Objetivo RLHF-PPO

$$J(\theta) = \mathbb{E}_{x,y \sim \pi_\theta} [r_\phi(x, y) - \beta \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})]$$

Clip Surrogate do PPO

$$L^{\text{CLIP}} = \mathbb{E}_t [\min(r_t A_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon) A_t)]$$

onde $r_t = \pi_\theta / \pi_{\text{old}}$ e A_t é a vantagem.

Papel do coeficiente KL β

β pequeno	Liberdade para maximizar reward; risco de <i>reward hacking</i>
β grande	Permanece próximo do SFT; pode não melhorar o alinhamento

Instabilidades típicas:

- *Reward hacking*: o modelo “engana” o RM
- *Mode collapse*: baixa diversidade de respostas
- *KL explosion*: divergência

Overview

1. Pré-treinamento
2. Fine-tuning Supervisionado (SFT)
3. RLHF com PPO
- 4. Direct Preference Optimization (DPO)**
5. Outros Métodos de Alinhamento
6. Ajuste Eficiente em Parâmetros (PEFT)
7. Quantização
8. Exercícios Práticos: PEFT e Quantização

DPO: Motivação

Problema com RLHF-PPO:

- Requer treinamento de um *Reward Model* separado
- Otimização RL é instável e sensível a hiperparâmetros
- Precisa manter 4 modelos em memória simultaneamente (política, referência, RM, *value function*)
- Alto custo computacional e de engenharia

Ideia do DPO (Rafailov et al., 2023):

Eliminar o RM explícito reparametrizando o problema de otimização diretamente em termos da política.

O DPO mostra que a solução ótima do RLHF

Comparação de infraestrutura

Componente	RLHF	DPO
Reward Model	✓	×
Value Function	✓	×
Política de referência	✓	✓
Geração online	✓	×
Dados de comparação	✓	✓

Derivação da Função de Perda do DPO

Da solução ótima do RLHF, pode-se mostrar que:

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Substituindo no modelo de preferência de Bradley-Terry e eliminando r^* :

Perda DPO

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

Interpretação intuitiva

A perda do DPO aumenta a probabilidade de y_w (escolhida) e diminui a de y_l (rejeitada), **ponderado pela mudança relativa em relação à referência**.

Gradiente implícito

O DPO aumenta a probabilidade relativa de y_w mais agressivamente quando o RM implícito **subestima** y_w , e vice-versa.

Datasets: UltraFeedback (64k), Anthropic-HH (170k), TL;DR Summarization (93k) — todos no formato (prompt, chosen, rejected)

DPO vs. PPO e Variantes do DPO

DPO vs. PPO

Aspecto	PPO	DPO
Complexidade	Alta	Baixa
Estabilidade	Baixa	Alta
Geração <i>online</i>	Sim	Não
<i>Reward Model</i>	Sim	Não
Distribuição	<i>On-policy</i>	<i>Off-policy</i>
Exploração	Sim	Não

Variantes do DPO

- **IPO** (Azar et al., 2023): regularização diferente, evita *overfitting* em preferências determinísticas
- **cDPO**: DPO com *label smoothing* para pares ruidosos
- **Robust DPO**: margem adaptativa baseada na confiança da preferência
- **ORPO**: sem modelo de referência, penaliza diretamente respostas rejeitadas

Overview

1. Pré-treinamento
2. Fine-tuning Supervisionado (SFT)
3. RLHF com PPO
4. Direct Preference Optimization (DPO)
- 5. Outros Métodos de Alinhamento**
6. Ajuste Eficiente em Parâmetros (PEFT)
7. Quantização
8. Exercícios Práticos: PEFT e Quantização

GRPO: *Group Relative Policy Optimization*

Proposto no DeepSeek-R1 (DeepSeek-AI, 2025), o GRPO elimina a *value function* do PPO substituindo-a por uma **estimativa de baseline por grupo**.

Objetivo GRPO

$$J_{\text{GRPO}} = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \hat{A}_i \cdot \min(r_i, \text{clip}(r_i, 1-\varepsilon, 1+\varepsilon)) \right]$$

onde $\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\})}{\text{std}(\{r_j\})}$ e G é o tamanho do grupo de amostras.

Usado para raciocínio matemático: reward

Vantagens do GRPO

- Sem *value function*: menos parâmetros, menos memória
- Estimativa de vantagem simples e estável
- Funciona bem com *reward* esparsos (verificação matemática)
- Permite raciocínio de *chain-of-thought* emergente

Resultado: DeepSeek-R1 alcança desempenho competitivo com OpenAI o1 em AIME e MATH usando GRPO com verificação automática de respostas.

SimPO e KTO

SimPO (Meng et al., 2024)

Motivação: DPO depende do modelo de referência, o que pode ser ineficiente.

Perda SimPO:

$$\mathcal{L}_{\text{SimPO}} = -\log \sigma \left(\frac{\beta}{|y_w|} \log \pi_{\theta}(y_w|x) - \frac{\beta}{|y_l|} \log \pi_{\theta}(y_l|x) - \gamma \right)$$

- Sem modelo de referência (π_{ref})
- *Length-normalized reward*: divide pelo comprimento da sequência para evitar viés de comprimento
- Margem γ garante separação entre

KTO (Ethayarajh et al., 2024)

Motivação: nem sempre temos pares (chosen, rejected); às vezes apenas sinal bom/ruim.

Perda KTO:

$$\mathcal{L}_{\text{KTO}} = \mathbb{E} \left[\lambda_w \sigma(\hat{r}_{\theta}(x, y_w) - z_0) + \lambda_l \sigma(z_0 - \hat{r}_{\theta}(x, y_l)) \right]$$

- Baseado em teoria de utilidade (Kahneman-Tversky)
- Apenas um sinal binário: **bom** ou **ruim**

Tabela Comparativa dos Métodos de Alinhamento

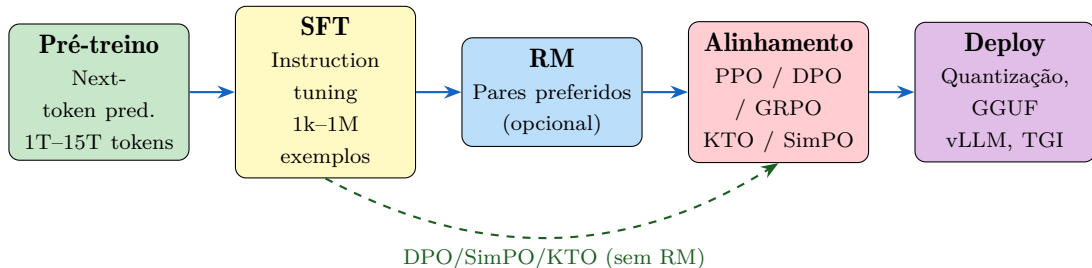
Método	Precisa de RM	Precisa de pares	Online	Sem ref.	Caso de uso típico
RLHF-PPO	✓	✓	✓	×	Alinhamento geral (InstructGPT)
DPO	×	✓	×	×	Alinhamento simples, eficiente
IPO	×	✓	×	×	DPO mais robusto a ruído
SimPO	×	✓	×	✓	Sem dependência de referência
KTO	×	×	×	×	Dados binários bom/ruim
GRPO	✓*	×	✓	×	Raciocínio matemático/código

* GRPO usa reward verificável automaticamente (ex.: checar resposta matemática), não um RM neural separado.

Tendência atual

Para a maioria dos casos, **SFT + DPO** é o pipeline padrão. PPO é reservado para cenários onde a geração *online* traz ganhos claros. GRPO cresce para tarefas de raciocínio com reward verificável.

Resumo: Pipeline Completo de Treinamento de LLMs



Pré-treino

GPT-3, LLaMA, Falcon
Datasets: The Pile, C4
Scaling laws de Chinchilla

SFT + Alinhamento

InstructGPT, ChatGPT
LLaMA-Chat, Mistral-Instruct
Qwen-Chat, DeepSeek-R1

Pesquisa ativa

DPO e variantes
GRPO para raciocínio
Constitutional AI (Anthropic)

Overview

1. Pré-treinamento
2. Fine-tuning Supervisionado (SFT)
3. RLHF com PPO
4. Direct Preference Optimization (DPO)
5. Outros Métodos de Alinhamento
- 6. Ajuste Eficiente em Parâmetros (PEFT)**
7. Quantização
8. Exercícios Práticos: PEFT e Quantização

Motivação: Custo do *Fine-tuning* Completo

Fazer *fine-tuning* completo de um LLM exige armazenar em GPU:

- **Pesos do modelo:** ~ 2 bytes/parâmetro (fp16)
- **Gradientes:** mesmo tamanho dos pesos
- **Estados do otimizador** (AdamW): ~ 12 bytes/parâmetro
- **Ativações:** proporcional ao *batch size*

Exemplo: modelo de 7B parâmetros

Pesos + gradientes + otimizador $\approx$ **112 GB** — fora do alcance de GPUs de consumo (RTX 4090: 24 GB).

Memória por parâmetro

Componente	fp32	fp16/bf16
Pesos	4 bytes	2 bytes
Gradientes	4 bytes	2 bytes
Adam m (1ª mom.)	4 bytes	—
Adam v (2ª mom.)	4 bytes	—
Total	16 bytes	~ 16 bytes*

* estados Adam mantidos em fp32 mesmo com pesos em fp16

Solução PEFT: treinar apenas uma fração pequena dos parâmetros, mantendo o modelo base congelado.

LoRA: *Low-Rank Adaptation*

Hu et al. (2022) propõem decompor a atualização de pesos em um produto de matrizes de **baixo rank**:

Decomposição LoRA

Para uma camada $W_0 \in \mathbb{R}^{d \times k}$ congelada, adicionamos um desvio treinável:

$$h = W_0 x + \frac{\alpha}{r} B A x$$

onde $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$ e α é um fator de escala.

- A inicializado com distribuição normal; B com zeros
- Somente A e B são atualizados durante o treino

Configurações típicas

Hiperparâmetro	Valor típico	Notas
<i>rank</i> r	4–64	Menor = menos parâmetros
α	$r-2r$	Controla a escala
<i>lora_dropout</i>	0.05	Regularização
Camadas alvo	Q, V	Projeções de atenção

Exemplo: LLaMA-7B com $r=16$, adaptando Q e V — apenas 0,17% dos parâmetros são treináveis, redução de memória de $\sim 3\times$.

Adaptar também as projeções de saída (O, *gate*, *up*, *down*) melhora o desempenho ao custo de mais parâmetros.

QLoRA: LoRA com Base Quantizada em 4 Bits

Dettmers et al. (2023) combinam LoRA com quantização do modelo base, permitindo ajustar modelos de 65B em uma única GPU de 48 GB.

Componentes chave do QLoRA

1. **NF4** (*NormalFloat 4-bit*): tipo de dado otimizado para pesos de redes neurais com distribuição normal; minimiza o erro de quantização em relação ao INT4 uniforme
2. **Dupla quantização**: quantizar também as constantes de quantização, economizando $\approx 0,37$ bits/parâmetro extra
3. **Paged Optimizers**: usa memória

Comparação de memória (modelo 7B)

Método	VRAM (treino)	Precisão
Full FT (fp16)	~ 112 GB	fp16
LoRA (fp16)	~ 40 GB	fp16
QLoRA (NF4)	~ 10 GB	NF4 + fp16

Custo: pequena degradação de qualidade ($\approx 0,1-0,2$ pontos em benchmarks) em troca de acessibilidade em hardware de consumo.

Implementação: bibliotecas `bitsandbytes` + `peft` do HuggingFace.

Prefix Tuning e Prompt Tuning

Prefix Tuning (Li & Liang, 2021)

Adiciona vetores treináveis como **prefixo** nas camadas de atenção do transformador:

- Prefixos inseridos no espaço de *keys* e *values* de **cada camada** do transformador
- Modelo base completamente congelado
- Parâmetros treináveis: $\sim 0,1\%$ do modelo
- Funciona bem para geração (tarefas seq2seq)

Desvantagem: otimização instável:

Prompt Tuning (Lester et al., 2021)

Versão simplificada: adiciona tokens **soft** apenas na camada de **entrada**:

- *Soft prompts*: vetores contínuos (não tokens discretos)
- Somente a camada de *embedding* é estendida
- Parâmetros: ~ 100 – 1000 vetores (pouquíssimos)
- Eficaz em escala: modelos $\geq 10B$; falha em modelos menores

Vantagem: um único modelo base com múltiplos *soft prompts* para diferentes tarefas

Comparação dos Métodos PEFT

Método	Params. treináveis	VRAM (7B)	Desempenho	Complexidade	Caso de uso típico
Full FT	100%	~112 GB	Máximo	Baixa	Dados abundantes, hardware dedicado
LoRA	0,1–1%	~40 GB	Alto	Baixa	Ajuste geral, produção
QLoRA	0,1–1%	~10 GB	Alto*	Média	Hardware de consumo (GPU única)
Prefix T.	<0,1%	~14 GB	Médio	Alta	Geração, tarefas seq2seq
Prompt T.	<0,01%	~14 GB	Médio**	Baixa	Modelos $\geq 10B$, múltiplas tarefas

* Pequena degradação vs. LoRA em fp16

** Eficaz apenas em modelos grandes

Recomendação prática

Para a maioria dos cenários, **QLoRA** é o padrão para ajuste em hardware limitado; **LoRA em bf16** quando há mais VRAM disponível. *Prompt tuning* é útil para servir muitas tarefas com um único modelo base.

Overview

1. Pré-treinamento
2. Fine-tuning Supervisionado (SFT)
3. RLHF com PPO
4. Direct Preference Optimization (DPO)
5. Outros Métodos de Alinhamento
6. Ajuste Eficiente em Parâmetros (PEFT)
- 7. Quantização**
8. Exercícios Práticos: PEFT e Quantização

Motivação: Reduzir o Custo de Inferência

Mesmo após o treino, LLMs são caros para servir em produção:

- Um modelo de 70B em fp16 requer **140 GB de VRAM** (2–4 GPUs A100/H100)
- A transferência de pesos domina a latência de inferência (*memory-bound* em decodificação autoregressiva)
- Objetivo: caber em hardware de consumo ou reduzir custos de nuvem

O que é quantização?

Representar pesos (e/ou ativações) com **menos bits**, reduzindo memória e acelerando operações matriciais:

$$W \rightarrow \frac{1}{2}(\max(W) + \min(W)) + \frac{W - \min(W)}{\max(W) - \min(W)} \cdot (\max(W) - \min(W))$$

Tamanho por precisão (modelo 7B)

Precisão	Bits	Tamanho
fp32	32	28 GB
fp16/bf16	16	14 GB
INT8	8	7 GB
INT4	4	3,5 GB
INT3	3	2,6 GB
INT2	2	1,8 GB

Quantização em 4 bits cabe em uma GPU de consumo (8 GB VRAM) ou roda inteiramente em GPU com RAM

Post-Training Quantization (PTQ)

LLM.int8() (Dettmers, 2022)

- Decompõe a multiplicação matricial: valores **salientes** (outliers) ficam em fp16; os demais são processados em INT8
- Sem perda perceptível de qualidade em modelos $\geq 6,7B$
- Implementado via biblioteca `bitsandbytes`

AWQ (Lin et al., 2023)

- *Activation-Aware Weight Quantization*
- Protege os **1% de pesos mais salientes** via escalonamento, evitando grande erro de quantização
- Supera GPTQ em modelos de instrução e código
- Suporte nativo em vLLM e TGI

GPTQ (Frantar et al., 2022)

- Quantização em INT4/INT3 baseada no **Hessiano de segunda ordem** (OBQ/OBC)

Resumo comparativo PTQ

Método	Bits	Qualidade	Velocidade
LLM.int8()	8	Alta	Média
GPTQ	3–4	Alta	Alta

GGUF e *llama.cpp*: Inferência Local em CPU

O formato **GGUF** (*GPT-Generated Unified Format*) é o padrão de facto para inferência de LLMs em hardware de consumo sem GPU.

Características do GGUF

- Arquivo único com pesos, tokenizador e metadados (substitui o formato antigo GGML)
- Suporta **quantização mista**: camadas com diferentes precisões (ex.: Q4_K_M, Q5_K_S, Q8_0)
- Inferência otimizada em CPU via *llama.cpp* (AVX-512, NEON, Metal/MPS para Mac)
- **Offload parcial para GPU**: parte das

Nomenclatura dos tipos GGUF

Tipo	Bits/param	Observação	
Q2_K	2,6	Qualidade baixa	muito
Q4_K_M	4,8	Equilíbrio	recomendado
Q5_K_M	5,7	Alta qualidade,	mais RAM
Q6_K	6,6	Quase sem perda	
Q8_0	8,5	Praticamente sem perda	

Exemplo prático: LLaMA-3-8B-Q4_K_M
≈ 4,7 GB — roda em um notebook com 8 GB de RAM a ~10–15 tokens/s.

Trade-offs: Perplexidade vs. Compressão

A **perplexidade** (*perplexity*) mede a qualidade do modelo na predição de tokens — valores menores são melhores:

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(x_t | x_{<t})\right)$$

Observações empíricas:

- **INT8:** <1% de aumento de PPL — sem degradação prática
- **INT4 (GPTQ/AWQ):** 2–5% em modelos pequenos; <2% em modelos $\geq 13\text{B}$
- **INT3 e abaixo:** degradação significativa; evitar exceto em dispositivos muito limitados

PPL vs. compressão — LLaMA-2-7B (WikiText-2)

Método	Tamanho	PPL	ΔPPL
fp16 (base)	13,5 GB	5,47	—
GPTQ-8b	6,7 GB	5,51	+0,7%
GPTQ-4b	3,9 GB	5,69	+4,0%
AWQ-4b	3,9 GB	5,63	+2,9%
GGUF Q4_K_M	4,1 GB	5,66	+3,5%
GPTQ-3b	3,0 GB	6,20	+13,4%

Regra prática

Use **INT4 (AWQ ou Q4_K_M)** como ponto de equilíbrio. Evite precisão abaixo de 4 bits exceto em dispositivos extremamente limitados

Overview

1. Pré-treinamento
2. Fine-tuning Supervisionado (SFT)
3. RLHF com PPO
4. Direct Preference Optimization (DPO)
5. Outros Métodos de Alinhamento
6. Ajuste Eficiente em Parâmetros (PEFT)
7. Quantização
- 8. Exercícios Práticos: PEFT e Quantização**

Exercícios Práticos: Roteiro

Objetivo: implementar PEFT e quantização em LLMs reais usando a Hugging Face e `llama.cpp`.

Exercícios desta seção

1. **Ex. 1 — LoRA:** configurar e treinar adaptadores de baixo *rank* com a biblioteca `peft`
2. **Ex. 2 — QLoRA:** combinar LoRA com base quantizada NF4 (`bitsandbytes`)
3. **Ex. 3 — Inferência INT8:** carregar modelo em 8 bits e gerar texto
4. **Ex. 4 — llama.cpp:** converter para GGUF, quantizar e executar localmente em CPU

Instalação das dependências

```
1 # Instalar bibliotecas necessarias
2 pip install torch transformers
3 pip install peft datasets trl
4 pip install bitsandbytes accelerate
```

Hardware mínimo

- **Ex. 1–3:** GPU com ≥ 6 GB VRAM (ex.: RTX 3060, T4 no Google Colab gratuito)
- **Ex. 4 (llama.cpp):** apenas CPU com ≥ 8 GB de RAM (sem GPU necessária)

Exercício 1 — Configurar LoRA com peft

Passos:

1. Carregar modelo base e tokenizador
2. Definir LoraConfig com rank r , α e módulos-alvo
3. Aplicar get_peft_model: congela pesos e injeta adaptadores A , B
4. Verificar percentual de parâmetros treináveis

Experimente

Varie r entre 4 e 64 e observe o impacto no número de parâmetros treináveis e na VRAM. Qual o efeito de incluir também "k_proj" e

```
1 import torch
2 from transformers import (
3     AutoModelForCausalLM, AutoTokenizer)
4 from peft import (
5     LoraConfig, get_peft_model, TaskType)
6
7 model = AutoModelForCausalLM.from_pretrained(
8     "facebook/opt-125m",
9     torch_dtype=torch.float16)
10 tokenizer = AutoTokenizer.from_pretrained(
11     "facebook/opt-125m")
12
13 lora_cfg = LoraConfig(
14     task_type=TaskType.CAUSAL_LM,
15     r=16,
16     lora_alpha=32,
17     lora_dropout=0.05,
18     target_modules=["q_proj", "v_proj"])
19
20 model = get_peft_model(model, lora_cfg)
21 model.print_trainable_parameters()
22 # trainable: 786,432 || total: 126,053,376
23 # 0.62% de parâmetros treinados
```


Exercício 1 — Treinar com SFTTrainer

O SFTTrainer da biblioteca trl aplica automaticamente a máscara de prompt (só calcula a perda nos tokens da resposta) e suporta *chat templates*.

Parâmetros chave do SFT com LoRA

- `learning_rate`: $10^{-4} - 3 \times 10^{-4}$ (maior que no fine-tuning completo)
- `num_train_epochs`: 1–3 épocas
- `gradient_accumulation_steps`: aumenta o batch efetivo sem custo extra de VRAM

Resultado: apenas os adaptadores LoRA

```
1 from trl import SFTTrainer
2 from transformers import TrainingArguments
3 from datasets import load_dataset
4
5 dataset = load_dataset(
6     "timdettmers/openassistant-guanaco",
7     split="train[:500]")
8
9 args = TrainingArguments(
10     output_dir="./output",
11     num_train_epochs=1,
12     per_device_train_batch_size=4,
13     gradient_accumulation_steps=4,
14     learning_rate=2e-4,
15     fp16=True,
16     logging_steps=10)
17
18 trainer = SFTTrainer(
19     model=model,
20     args=args,
21     train_dataset=dataset,
22     dataset_text_field="text",
23     max_seq_length=256)
```

Exercício 2 — QLoRA: Base Quantizada em NF4

Objetivo: repetir o Exercício 1 com o modelo base quantizado em 4 bits (NF4), viabilizando o ajuste de modelos de 7B–13B em uma GPU de consumo.

BitsAndBytesConfig: opções NF4

- `load_in_4bit`: ativa quantização NF4
- `bnb_4bit_use_double_quant`: dupla quantização ($\approx 0,37$ bits/param a menos)
- `bnb_4bit_quant_type="nf4"`: distribuição normal $\Rightarrow$ menor erro de quantização
- `bnb_4bit_compute_dtype`:

```
1 import torch
2 from transformers import (
3     AutoModelForCausalLM, BitsAndBytesConfig)
4 from peft import get_peft_model, LoraConfig
5
6 bnb_cfg = BitsAndBytesConfig(
7     load_in_4bit=True,
8     bnb_4bit_use_double_quant=True,
9     bnb_4bit_quant_type="nf4",
10    bnb_4bit_compute_dtype=torch.bfloat16)
11
12 # Modelo base quantizado em NF4
13 model = AutoModelForCausalLM.from_pretrained(
14     "facebook/opt-125m",
15     quantization_config=bnb_cfg,
16     device_map="auto")
17
18 # Aplicar LoRA sobre a base quantizada
19 lora_cfg = LoraConfig(r=16, lora_alpha=32,
```

Exercício 3 — Inferência com Quantização INT8

Objetivo: carregar um modelo quantizado em 8 bits para **inferência** (sem treino), reduzindo a VRAM à metade sem degradação perceptível de qualidade.

LLM.int8() na prática

- Pesos **salientes** (outliers) ficam em fp16
- Demais pesos em INT8 — sem queda de qualidade
- Compatível com `device_map="auto"` para distribuição em múltiplas GPUs

```
1 import torch
2 from transformers import (
3     AutoModelForCausalLM, AutoTokenizer)
4
5 MODEL = "facebook/opt-125m"
6
7 # Carregar em INT8 (LLM.int8())
8 model = AutoModelForCausalLM.from_pretrained(
9     MODEL,
10    load_in_8bit=True,
11    device_map="auto")
12
13 tokenizer = AutoTokenizer.from_pretrained(
14     MODEL)
15
16 # Gerar texto com o modelo quantizado
17 prompt = "Redes neurais sao"
18 inputs = tokenizer(
19    prompt, return_tensors="pt").to("cuda")
20
```

Exercício 4 — GGUF e llama.cpp: CPU sem GPU

Objetivo: converter um modelo Hugging Face para GGUF, quantizá-lo para Q4_K_M e executar inferência inteiramente em CPU.

Fluxo de trabalho

1. Compilar llama.cpp (ou usar **Ollama** como alternativa simplificada)
2. Converter modelo HF → GGUF (fp16)
3. Quantizar GGUF para Q4_K_M ($\approx 4,8$ bits/param)
4. Executar via CLI ou servidor OpenAI-compatível

```
1 # 1. Clonar e compilar llama.cpp
2 git clone
  ↪ https://github.com/ggerganov/llama.cpp
3 cd llama.cpp
4 cmake -B build && cmake --build build -j8
5
6 # 2. Converter modelo HF para GGUF (fp16)
7 python convert_hf_to_gguf.py \
8     meta-llama/Llama-3.2-1B \
9     --outfile llama-3.2-1b-fp16.gguf
10
11 # 3. Quantizar para Q4_K_M
12 ./build/bin/llama-quantize \
13     llama-3.2-1b-fp16.gguf \
14     llama-3.2-1b-Q4_K_M.gguf Q4_K_M
15
16 # 4. Inferencia via CLI
17 ./build/bin/llama-cli \
18     -m llama-3.2-1b-Q4_K_M.gguf \
```

Questões para Reflexão

Sobre PEFT / LoRA

1. Por que A é inicializado com distribuição normal e B com zeros? O que aconteceria se ambos fossem aleatórios?
2. Aumentar o rank r de 4 para 64 sempre melhora o desempenho final? Qual o *trade-off*?
3. Por que os adaptadores LoRA no QLoRA ficam em bf16 e não em NF4, assim como o modelo base?
4. Como você fundiria os adaptadores LoRA de volta ao modelo base (`merge_and_unload`) para eliminar

Sobre Quantização

5. Qual a diferença prática entre LLM.int8() e GPTQ-4b em termos de velocidade de inferência e qualidade?
6. Por que a quantização degrada menos modelos maiores (ex.: 70B) do que modelos menores (ex.: 1B)?
7. Quando você escolheria Q4_K_M em vez de Q8_0 no formato GGUF? Justifique em termos de qualidade, velocidade e RAM disponível.
8. Um modelo quantizado em INT4 pode ser re-treinado diretamente